LLVM BOLT 中的自适应 CMOV 优化

系统优化课程设计说明 · 简洁版 · 2026 年 11 月 13 日

基线：2026 年 11 月 2 日 LLVM main；优化选项：-cmov-branchify-runtime，默认关闭。

1 问题描述

CMOV 是按条件选择寄存器值的指令。它可以减少分支预测失败，但当条件长期偏向一侧时，改成条件分支可能更快。离线采样得到的是训练输入的情况，实际输入变化后，原来的选择也可能不再合适。

本作业在 BOLT 中同时保留原 CMOV 实现和分支实现，先筛选有机会的代码，再根据当前任务的耗时选择版本。对空表等能直接确定结果的任务，在入口直接完成，避免为观察性能付出额外成本。

2 优化设计

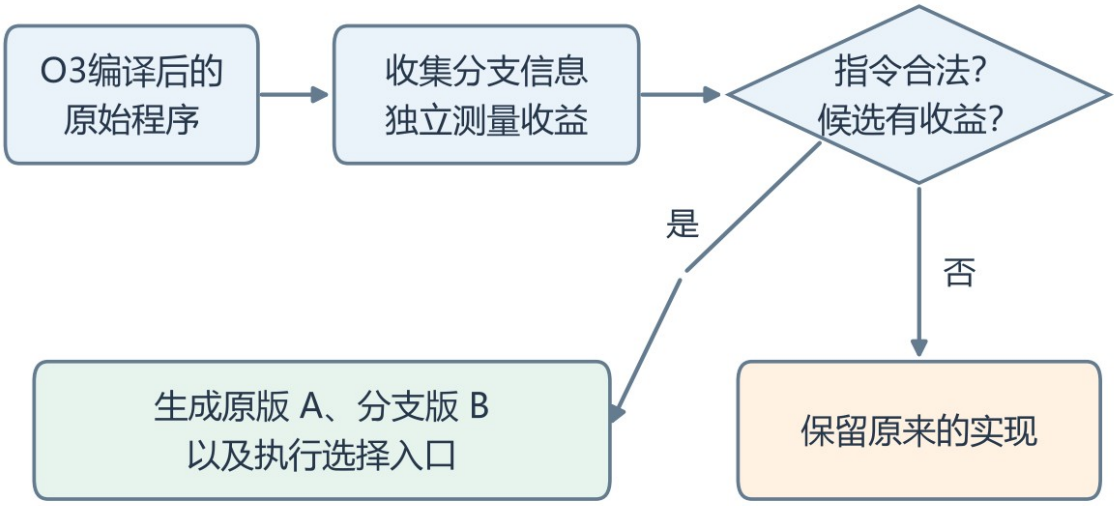


图 1 生成优化程序的流程

原版记为 A，分支版记为 B。核心思路有三点：先确认候选值得优化；运行时继续观察收益；确认结果为 0 的任务直接完成。LLVM 已有基础 CMOV 转分支方法，本项目的新增工作是 BOLT 中的组级改写、双版本选择和批次入口处理。

检查项	当前条件
指令	循环内 2～4 条同条件、纯寄存器 CMOV，局部编码不增长
采样与校准	至少 10000 个样本、偏置至少 95%；独立校准收益比区间下界≥1.02
程序接口	有明确批次入口；共享输入只读；内核为受支持的叶函数

2.1 运行时如何选择

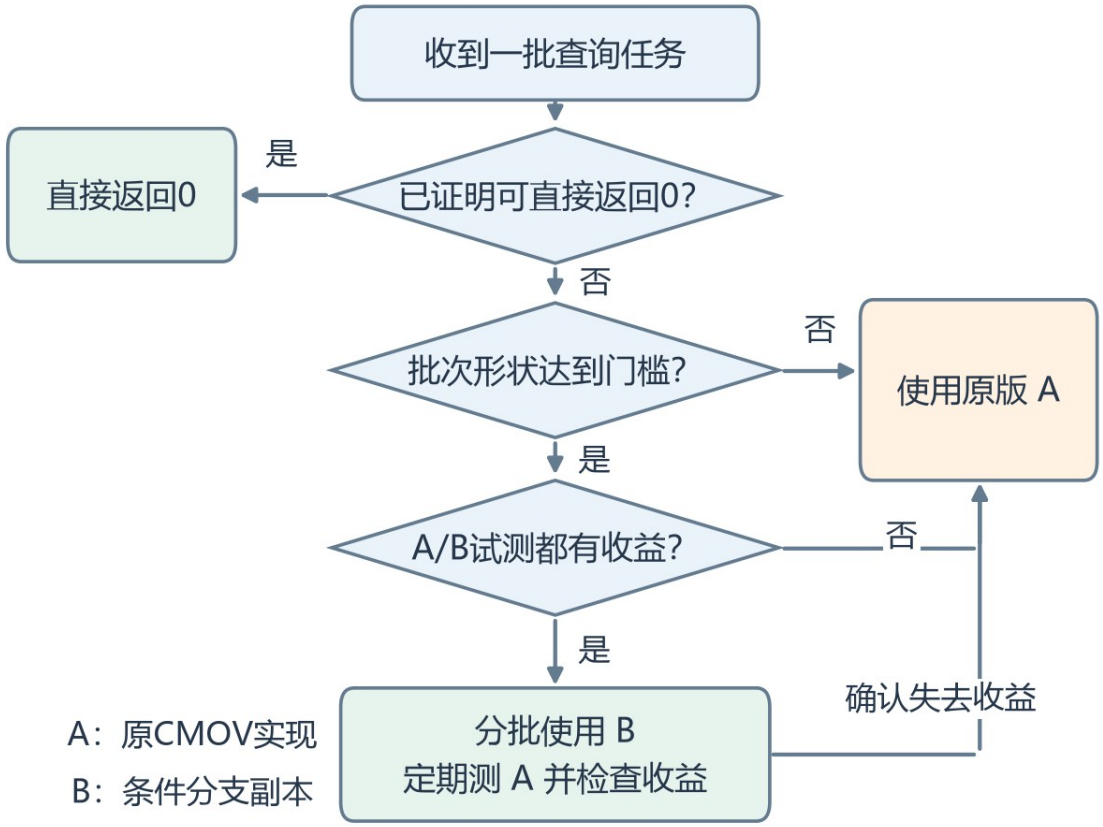


图 2 一次调用内的执行选择

普通任务要求至少 512 批、每批 4096 条查询。准入时最多比较 3 对 A/B，三对都显示 B 快至少 10%才使用 B；之后每两批 B 合并计时，并定期测 A。收益变小时重新测 A 确认，确认后本次调用剩余工作使用 A。

3 实现细节

部分	做了什么
BOLT 变换	识别相邻同条件 CMOV，生成分支副本，并维护控制流和展开信息。
执行入口	检查可直接得到 0 的条件；跳过结果为 0 的空查询前缀，再执行剩余任务。
运行库	负责 A/B 比较、持续检查和回退。时钟异常时从尚未处理的位置继续。

直接返回 0 的条件来自对真实指令和控制流的检查。字段偏移从指令中提取；遇到未知前缀、不同返回常量或不支持的指令时，保留原实现。空查询任务不会读取未使用的上下文。主要源码见 CMOVBranchify.cpp、X86MCPlusBuilder.cpp 和 runtime/cmov*.cpp。

4 实验验证

平台为 Ryzen 7 7735H、WSL2、Linux x86-64，绑定来宾逻辑 CPU 2。测试负载是 Clang 18 正常-O3 编译的批量 lower_bound 查找。基准和优化版使用相同输入、相同通用 BOLT 参数，通过选项开关作比较。

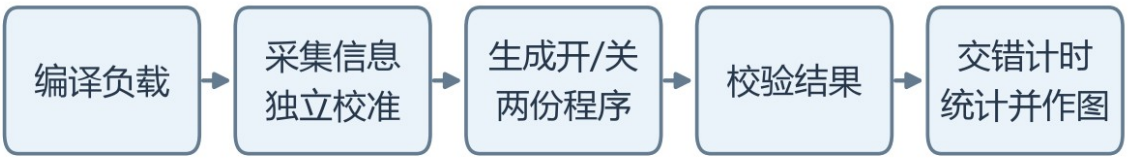


图 3 实验流程

训练和校准与正式测量分开。正式实验覆盖 52 项常规负载和 7 项边界，共 31 轮；每轮交错执行并检查输出。以每轮配对耗时比的中位数作为结果，图中误差线为逐项 95%区间。

4.1 有收益的负载

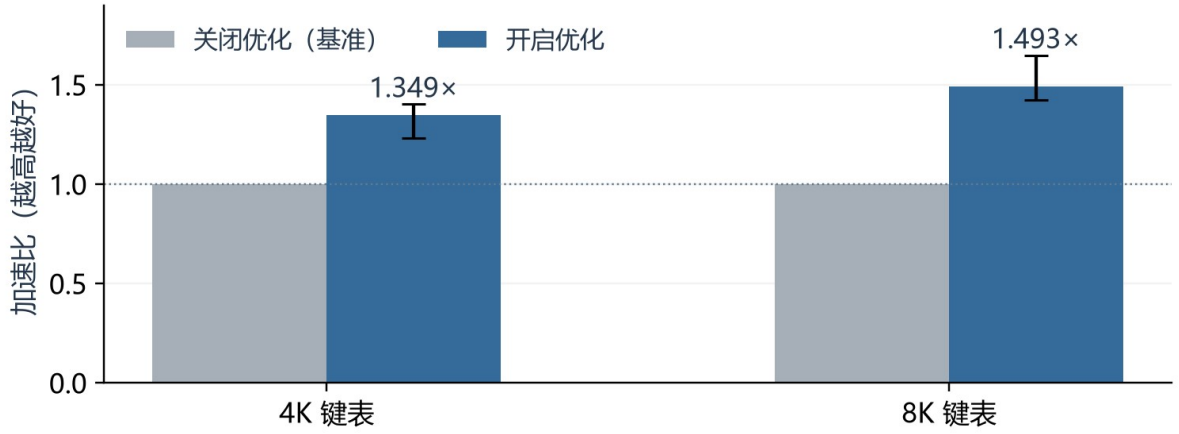


图 4 强偏置查找的加速比；越高越好

键表大小	开启后的加速比	95%区间
4K	1.349 倍	[1.232, 1.402]
8K	1.493 倍	[1.425, 1.648]

加速比=关闭选项耗时÷开启选项耗时。主要正例中，4K 和 8K 任务分别减少约 25.86%和 33.01%的耗时。强偏置和部分分布切换负载有收益；均匀输入、短调用和准入混淆输入用于检查保护成本。

4.2 无收益负载与空表

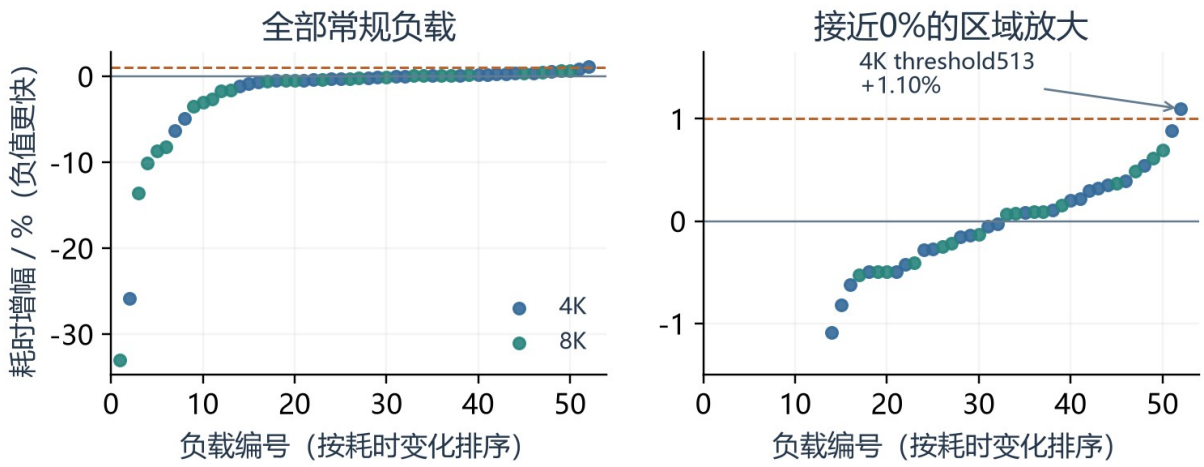


图 5 52 项常规负载的耗时变化；虚线为 1%参考线

常规负载中，51/52 项的中位回退低于 1%，最大为 4K threshold513 的 1.10%。独立长计时补测中，threshold512/513 分别回退约 0.17%和 0.21%，区间支持约 1%的参考。两套数据分别保存，主矩阵保持完整。

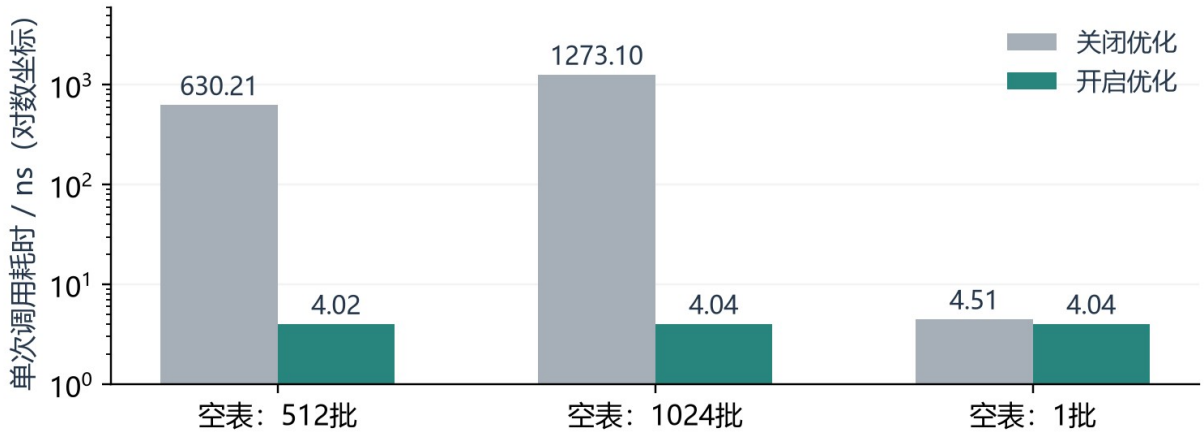


图 6 空表调用耗时中位数；纵轴使用对数坐标

空表 512 批和 1024 批调用开启后均约 4 ns，单批空表约 4.04 ns。大批空表的结果能整批确定为 0，因此省去了内核调用和计时。这个收益对应零工作路径；普通查找的效果见图 4。

4.3 正确性与复现

验证内容	结果
计时过程中的结果校验	正式 14632 次执行、独立长计时 992 次执行全部通过
独立功能对照	原始、关闭、开启三个版本，共 468 次运行结果一致
零工作路径	344 个组合通过；另检查两种匹配模式和两种拒绝模式
工具与运行库	32 项集成、28 项安全检查、755 项可运行 BOLT 回归通过
交付流程	Docker 中重新训练、校准并完成 8 项流程；校准拒绝产物另有 156 次功能运行通过

5 局限性

当前面向 Linux x86-64 和有明确批次入口的程序，内核只接受受支持的叶函数与指令模式。共享输入须在一次调用期间保持只读。其他模式保留原代码。

结果来自一台 WSL 机器；换 CPU 后需要重新采样、校准和计时。当前常规最差中位回退为 1.10%，还应继续扩展更小的非空任务及其他程序，验证保护成本。

运行方法

准备 Linux x86-64、Bash、Python3 和 Docker。仓库入口会下载并校验预编译镜像，也可从固定源码构建。

```
./run.sh                  # 快速流程
./run.sh full             # 完整负载与空表实验
./run.sh functional       # 功能和安全检查
./run.sh evidence         # 离线复算保存的数据
./run.sh build --source --jobs 2  # 从源码构建
```

提交文件与数据位置

内容	仓库位置
完整修改	stage_1/patches/cmov-runtime.patch
设计文档	stage_1/docs/design.docx、design.pdf、design.md
统计和原始记录	stage_1/evidence/final/、precision/
负载与脚本	stage_1/project/cmov_runtime/

源码固定提交为 3bd152ddbb6df45e0e07d6c9e9eb1573dcfd3dc6。基础 CMOV 转换已有 LLVM 实现，上游覆盖复核记录截至 2026 年 11 月 12 日。代码与文档整理使用 AI 辅助，实验数据由实际运行记录生成。